

Statistical and Machine Learning SS26

Solution Exercise Sheet 5

1)

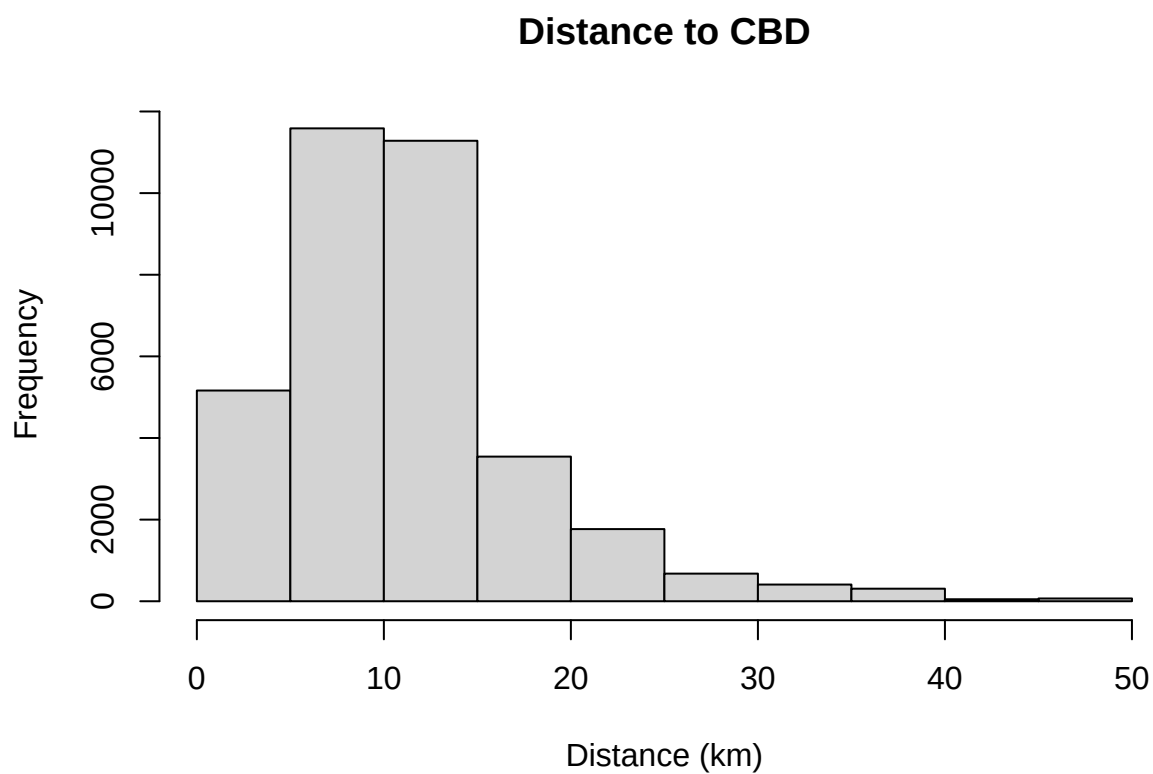
a) Load and inspect the dataset “Melbourne House Prices”.

```
melbdata <- read.csv("./Melbourne_housing_FULL.csv", header = TRUE) #load data
str(melbdata) #overview: number of obs&var, names of var
```

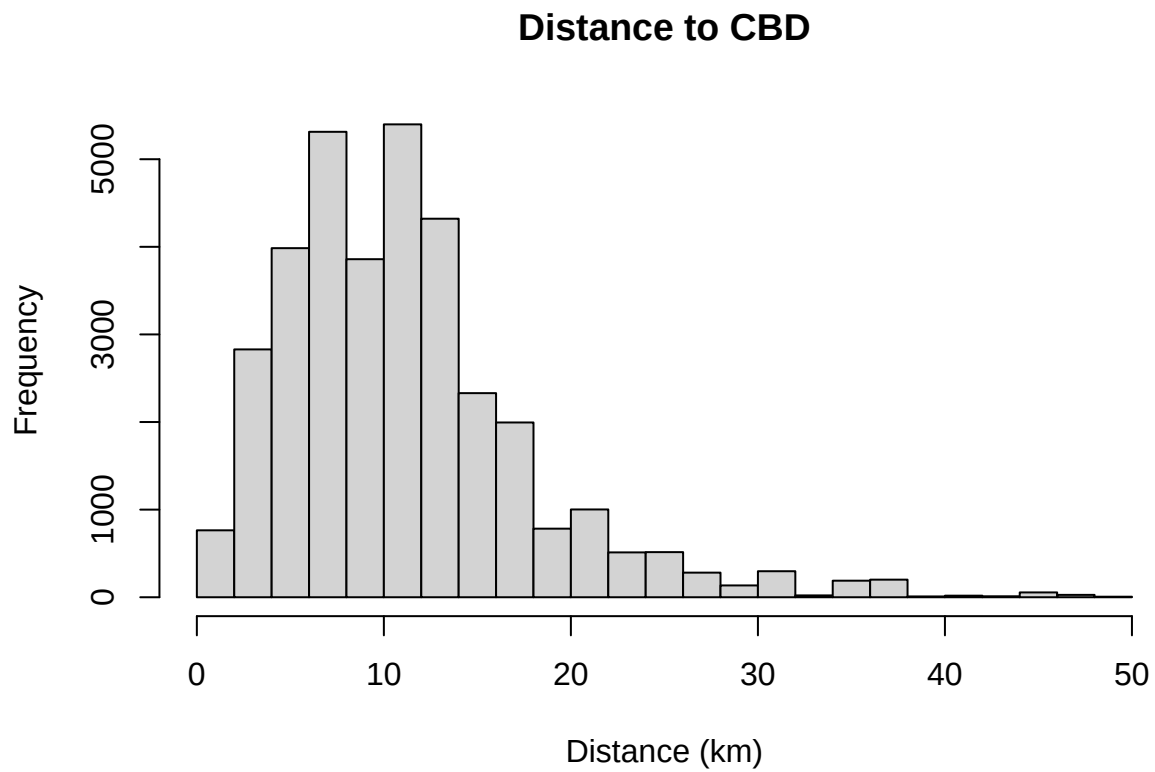
```
## 'data.frame':    34857 obs. of  21 variables:
## $ Suburb       : chr  "Abbotsford" "Abbotsford" "Abbotsford" "Abbotsford" ...
## $ Address      : chr  "68 Studley St" "85 Turner St" "25 Bloomburg St" "18/659 Victoria St" ...
## $ Rooms        : int   2 2 2 3 3 3 4 4 2 2 ...
## $ Type         : chr  "h" "h" "h" "u" ...
## $ Price        : int   NA 1480000 1035000 NA 1465000 850000 1600000 NA NA NA ...
## $ Method       : chr  "SS" "S" "S" "VB" ...
## $ SellerG      : chr  "Jellis" "Biggin" "Biggin" "Rounds" ...
## $ Date         : chr  "3/09/2016" "3/12/2016" "4/02/2016" "4/02/2016" ...
## $ Distance     : num   2.5 2.5 2.5 2.5 2.5 2.5 2.5 2.5 2.5 2.5 ...
## $ Postcode     : int   3067 3067 3067 3067 3067 3067 3067 3067 3067 3067 ...
## $ Bedroom2     : int   2 2 2 3 3 3 3 3 4 3 ...
## $ Bathroom     : int   1 1 1 2 2 2 1 2 1 2 ...
## $ Car          : int   1 1 0 1 0 1 2 2 2 1 ...
## $ Landsize     : int   126 202 156 0 134 94 120 400 201 202 ...
## $ BuildingArea : num   NA NA 79 NA 150 NA 142 220 NA NA ...
## $ YearBuilt    : int   NA NA 1900 NA 1900 NA 2014 2006 1900 1900 ...
## $ CouncilArea  : chr  "Yarra City Council" "Yarra City Council" "Yarra City Council" "Yarra City Council" ...
## $ Lattitude    : num  -37.8 -37.8 -37.8 -37.8 -37.8 ...
## $ Longitude    : num   145 145 145 145 145 ...
## $ Regionname   : chr  "Northern Metropolitan" "Northern Metropolitan" "Northern Metropolitan" "Northern Metropolitan" ...
## $ Propertycount: int   4019 4019 4019 4019 4019 4019 4019 4019 4019 4019 ...
```

b) Plot the histogram of the distance from the city center as frequency and as density.

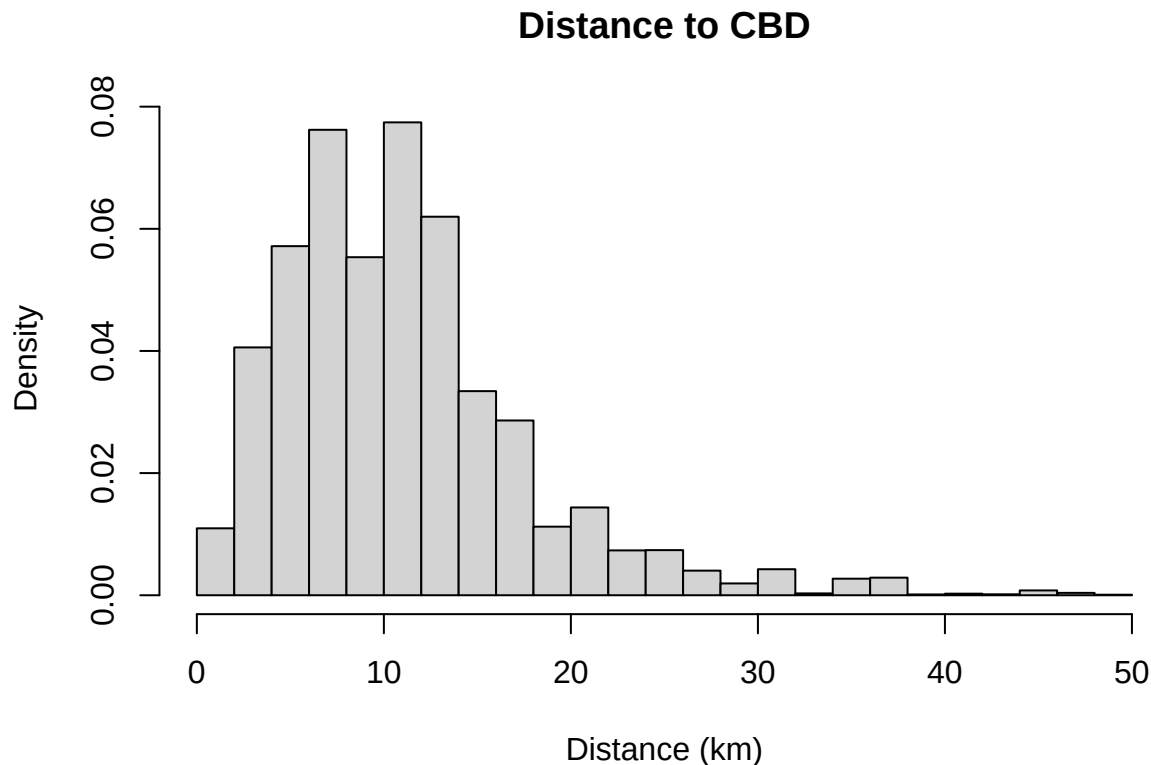
```
hist(na.omit(melbdata$Distance), main = "Distance to CBD", xlab = "Distance (km)")
```



```
#na.omit removes missing values; default: freq=TRUE -> frequency; label plot; label x-axis  
hist(na.omit(melbdata$Distance), breaks = 30,  
     main = "Distance to CBD", xlab = "Distance (km)" )
```



```
# uses more bins -> histogram becomes more detailed  
hist(na.omit(melbdata$Distance), breaks = 30, freq = FALSE,  
     main = "Distance to CBD", xlab = "Distance (km)")
```



#freq_FALSE -> density, area of all bars adds up to 1

- c) Estimate the density of the Distance variable using four different kernel functions: Gaussian, Epanechnikov, Triangular, and Biweight. Use a fixed bandwidth of $bw = 1$ for all kernels. Plot the histogram of Distance as a density (breaks = 50). Overlay all four KDE curves on the histogram in different colors.

Why is using kernel density useful? Smooth, continuous version of a histogram (which are blocky and depend heavily on where you put the bin edges)

```
d1 <- density(na.omit(melbdata$Distance), bw = 1, kernel = "gaussian")
d2 <- density(na.omit(melbdata$Distance), bw = 1, kernel = "epanechnikov")
d3 <- density(na.omit(melbdata$Distance), bw = 1, kernel = "triangular")
d4 <- density(na.omit(melbdata$Distance), bw = 1, kernel = "biweight")
#Each line estimates a smooth density curve from the same data, using a different kernel
# shape but the same bandwidth bw = 1.
# x axis: distance from city center; y axis: the estimated density at each point

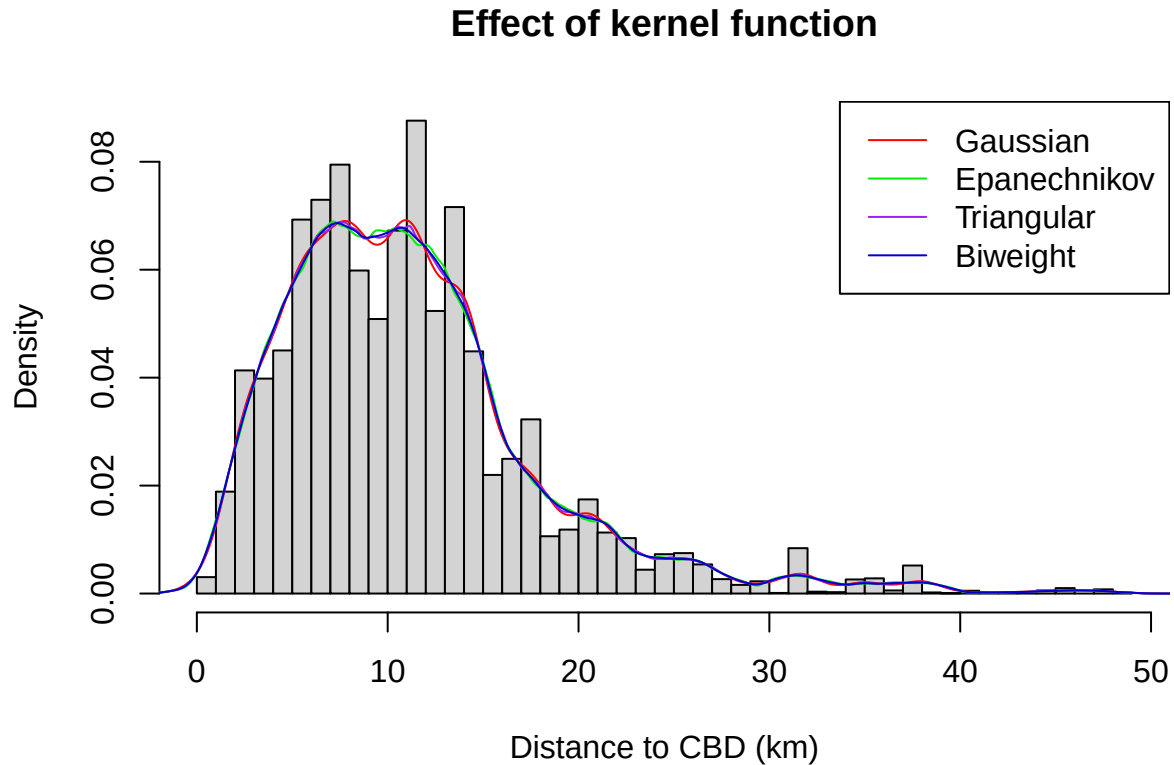
#Density at a point tells you how concentrated the data is there (not a probability!) it only
# becomes an actual probability when computing the area under the curve over an interval.

#putting together histogram and 4 density curves
hist(na.omit(melbdata$Distance), freq = FALSE, breaks = 50,
     main = "Effect of kernel function", xlab = "Distance to CBD (km)")
lines(d1$x, d1$y, col = "red") #takes x and y values of d1 (gaussian) and plots the curve
```

```

lines(d2$x, d2$y, col = "green2")
lines(d3$x, d3$y, col = "purple")
lines(d4$x, d4$y, col = "blue3")
legend("topright", c("Gaussian", "Epanechnikov", "Triangular", "Biweight"),
      lty = c(1, 1, 1, 1), col = c("red", "green2", "purple", "blue3"))

```



All four different kernel density curves look very similar!

- d) Refit and plot the KDE using only the Gaussian kernel with three different bandwidths: $bw = 0.5$, $bw = 1$, and $bw = 1.5$.

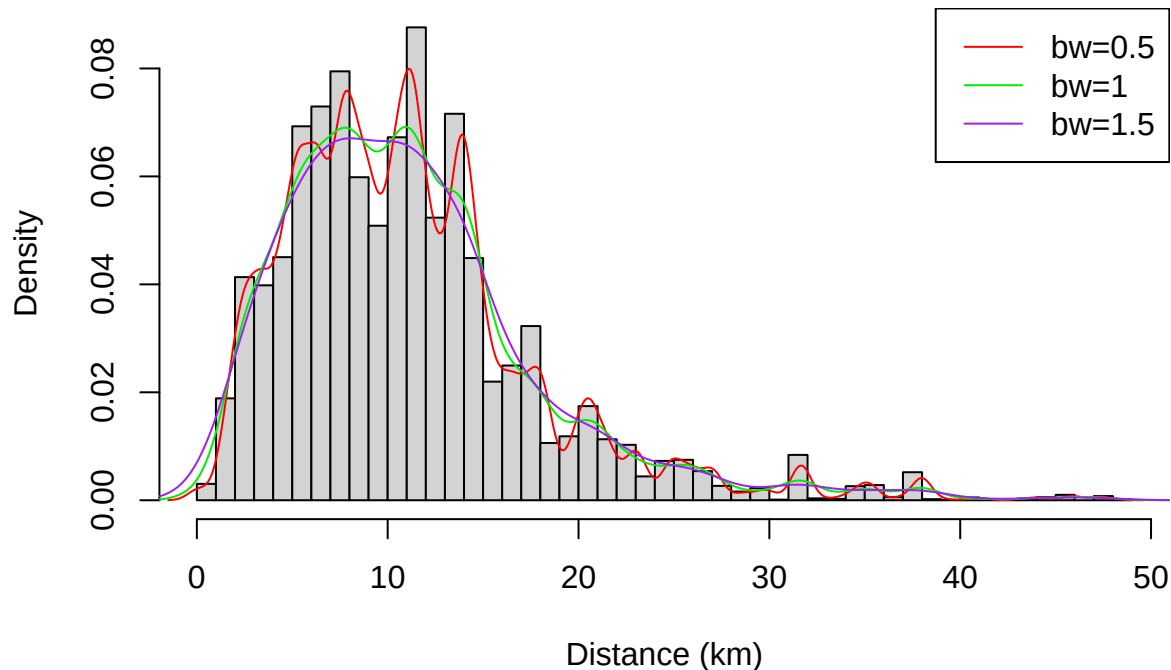
```

# only use gaussian kernel, different bandwidths
d1 <- density(na.omit(melbdata$Distance), bw = 0.5, kernel = "gaussian")
d2 <- density(na.omit(melbdata$Distance), bw = 1, kernel = "gaussian")
d3 <- density(na.omit(melbdata$Distance), bw = 1.5, kernel = "gaussian")

hist(na.omit(melbdata$Distance), freq = FALSE, breaks = 50, main = "Effect of bandwidth",
     xlab = "Distance (km)")
lines(d1$x, d1$y, col = "red")
lines(d2$x, d2$y, col = "green2")
lines(d3$x, d3$y, col = "purple")
legend("topright", c("bw=0.5", "bw=1", "bw=1.5"),
      lty=c(1,1,1,1), col=c("red", "green2", "purple"))

```

Effect of bandwidth



Choice of bandwidth is crucial, as it controls the degree of smoothing
 h too small: over-fitted, picks up noise
 h too large: over-smoothed, important features may disappear

Takeaway: choice of bandwidth is very important in KDE, whereas kernel choice barely mattered

- e) Use the density estimation to estimate the probability of finding a house for sale between 8 and 10km from the CBD

```
bw <- bw.bcv(na.omit(melbdata$Distance), lower = 0.01, upper = 10)
# Instead of manually picking bw = 1 like before, bw.bcv() automatically finds
# the optimal bandwidth using Biased Cross-Validation.
# it searches between 0.01 and 10 and picks the value that minimizes the estimation error.

d2 <- density(na.omit(melbdata$Distance), bw = bw, kernel = "gaussian")
# computes KDE using optimal bandwidth we just calculated
f <- approxfun(d2)
# d2 is just a table of values, approxfun turns these values into a function that
# we can call at any x value in the range and gives the estimated density at x km

#each of the following methods compute the same thing:
# the probability of finding a house for sale between 8 and 10km from the CBD
p <- integrate(f, 8, 10); p
```

```
## 0.1334915 with absolute error < 5.7e-06
```

```
#Directly integrates the function f between 8 and 10.
```

```
ind = which((d2$x>8)&(d2$x<10))  
mean(d2$y[ind])*(10-8)
```

```
## [1] 0.1334979
```

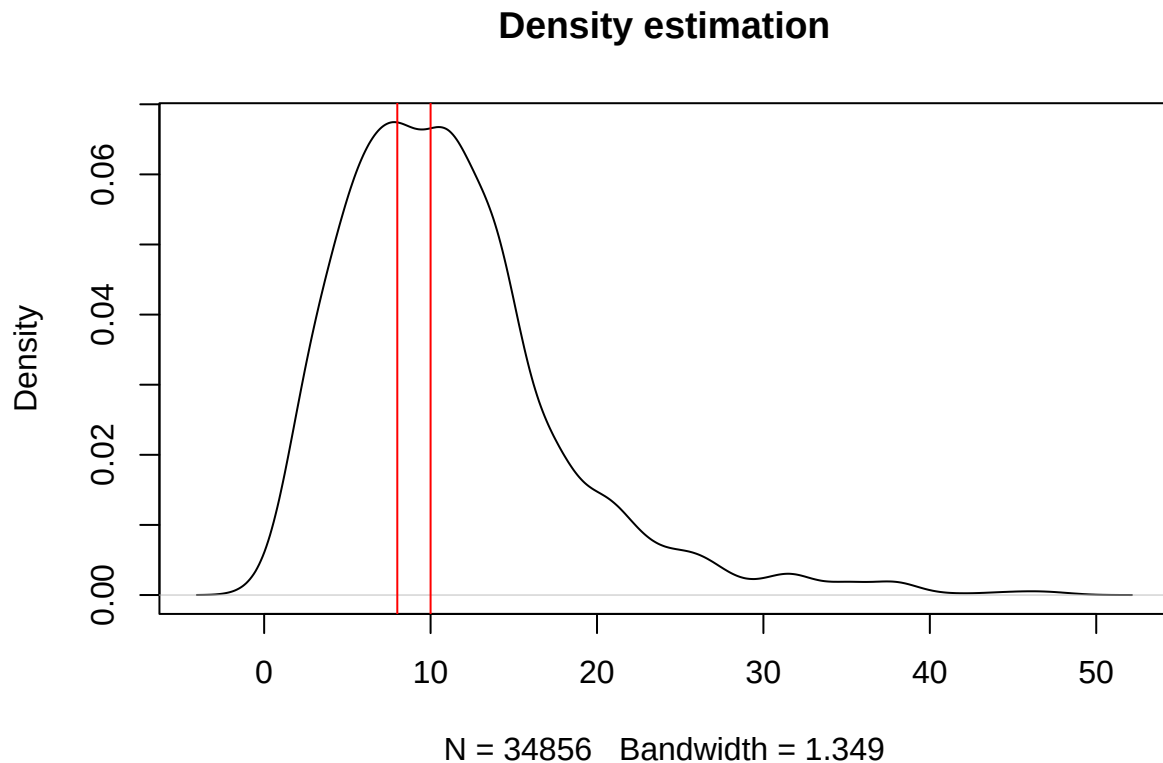
```
# Finds all grid points between 8 and 10, takes their average density,  
# multiplies by the interval width
```

```
mean(f(seq(8, 10, length.out = 100)))*(10-8)
```

```
## [1] 0.1334966
```

```
#Creates 100 evenly spaced points between 8 and 10, evaluates f at each,  
# takes the average and multiplies by interval width.  
# (More precise than Method 2 because it uses more points.)
```

```
plot(d2, main = "Density estimation")  
abline(v=8, col = "red")  
abline(v=10, col = "red")
```



```
bw
```

```
## [1] 1.348579
```

2)

a) Load and inspect the data

```
library(ISLR2) #load packages  
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
data("Caravan")  
attach(Caravan) #if we attach the data set, we don't have to write caravan$variable anymore  
str(Caravan)
```

```
## 'data.frame': 5822 obs. of 86 variables:  
## $ MOSTYPE : num 33 37 37 9 40 23 39 33 33 11 ...  
## $ MAANTHUI : num 1 1 1 1 1 1 2 1 1 2 ...  
## $ MGEMOMV : num 3 2 2 3 4 2 3 2 2 3 ...  
## $ MGEMLEEF : num 2 2 2 3 2 1 2 3 4 3 ...  
## $ MOSHOOFD : num 8 8 8 3 10 5 9 8 8 3 ...  
## $ MGODRK : num 0 1 0 2 1 0 2 0 0 3 ...  
## $ MGODPR : num 5 4 4 3 4 5 2 7 1 5 ...  
## $ MGODOV : num 1 1 2 2 1 0 0 0 3 0 ...  
## $ MGODGE : num 3 4 4 4 4 5 5 2 6 2 ...  
## $ MRELGE : num 7 6 3 5 7 0 7 7 6 7 ...  
## $ MRELSA : num 0 2 2 2 1 6 2 2 0 0 ...  
## $ MRELOV : num 2 2 4 2 2 3 0 0 3 2 ...  
## $ MFALLEEN : num 1 0 4 2 2 3 0 0 3 2 ...  
## $ MFGEKIND : num 2 4 4 3 4 5 3 5 3 2 ...  
## $ MFWEKIND : num 6 5 2 4 4 2 6 4 3 6 ...  
## $ MOPLHOOG : num 1 0 0 3 5 0 0 0 0 0 ...  
## $ MOPLMIDD : num 2 5 5 4 4 5 4 3 1 4 ...  
## $ MOPLLAAG : num 7 4 4 2 0 4 5 6 8 5 ...  
## $ MBERHOOG : num 1 0 0 4 0 2 0 2 1 2 ...  
## $ MBERZELF : num 0 0 0 0 5 0 0 0 1 0 ...  
## $ MBERBOER : num 1 0 0 0 4 0 0 0 0 0 ...  
## $ MBERMIDD : num 2 5 7 3 0 4 4 2 1 3 ...  
## $ MBERARBG : num 5 0 0 1 0 2 1 5 8 3 ...  
## $ MBERARBO : num 2 4 2 2 0 2 5 2 1 3 ...  
## $ MSKA : num 1 0 0 3 9 2 0 2 1 1 ...  
## $ MSKB1 : num 1 2 5 2 0 2 1 1 1 2 ...  
## $ MSKB2 : num 2 3 0 1 0 2 4 2 0 1 ...  
## $ MSKC : num 6 5 4 4 0 4 5 5 8 4 ...  
## $ MSKD : num 1 0 0 0 0 2 0 2 1 2 ...
```



```

## $ MHHUUR : num 1 2 7 5 4 9 6 0 9 0 ...
## $ MHKOOP : num 8 7 2 4 5 0 3 9 0 9 ...
## $ MAUT1 : num 8 7 7 9 6 5 8 4 5 6 ...
## $ MAUT2 : num 0 1 0 0 2 3 0 4 2 1 ...
## $ MAUTO : num 1 2 2 0 1 3 1 2 3 2 ...
## $ MZFONDS : num 8 6 9 7 5 9 9 6 7 6 ...
## $ MZPART : num 1 3 0 2 4 0 0 3 2 3 ...
## $ MINKM30 : num 0 2 4 1 0 5 4 2 7 2 ...
## $ MINK3045: num 4 0 5 5 0 2 3 5 2 3 ...
## $ MINK4575: num 5 5 0 3 9 3 3 3 1 3 ...
## $ MINK7512: num 0 2 0 0 0 0 0 0 0 1 ...
## $ MINK123M: num 0 0 0 0 0 0 0 0 0 0 ...
## $ MINKGEM : num 4 5 3 4 6 3 3 3 2 4 ...
## $ MKOOPKLA: num 3 4 4 4 3 3 5 3 3 7 ...
## $ PWAPART : num 0 2 2 0 0 0 0 0 0 2 ...
## $ PWABEDR : num 0 0 0 0 0 0 0 0 0 0 ...
## $ PWALAND : num 0 0 0 0 0 0 0 0 0 0 ...
## $ PPERSAUT: num 6 0 6 6 0 6 6 0 5 0 ...
## $ PBESAUT : num 0 0 0 0 0 0 0 0 0 0 ...
## $ PMOTSCO : num 0 0 0 0 0 0 0 0 0 0 ...
## $ PVRAAUT : num 0 0 0 0 0 0 0 0 0 0 ...
## $ PAANHANG: num 0 0 0 0 0 0 0 0 0 0 ...
## $ PTRACTOR: num 0 0 0 0 0 0 0 0 0 0 ...
## $ PWERKT : num 0 0 0 0 0 0 0 0 0 0 ...
## $ PBROM : num 0 0 0 0 0 0 0 3 0 0 ...
## $ PLEVEN : num 0 0 0 0 0 0 0 0 0 0 ...
## $ PPERSONG: num 0 0 0 0 0 0 0 0 0 0 ...
## $ PGEZONG : num 0 0 0 0 0 0 0 0 0 0 ...
## $ PWAOREG : num 0 0 0 0 0 0 0 0 0 0 ...
## $ PBRAND : num 5 2 2 2 6 0 0 0 0 3 ...
## $ PZEILPL : num 0 0 0 0 0 0 0 0 0 0 ...
## $ PPLEZIER: num 0 0 0 0 0 0 0 0 0 0 ...
## $ PFIETS : num 0 0 0 0 0 0 0 0 0 0 ...
## $ PINBOED : num 0 0 0 0 0 0 0 0 0 0 ...
## $ PBYSTAND: num 0 0 0 0 0 0 0 0 0 0 ...
## $ AWAPART : num 0 2 1 0 0 0 0 0 0 1 ...
## $ AWABEDR : num 0 0 0 0 0 0 0 0 0 0 ...
## $ AWALAND : num 0 0 0 0 0 0 0 0 0 0 ...
## $ APERSAUT: num 1 0 1 1 0 1 1 0 1 0 ...
## $ ABESAUT : num 0 0 0 0 0 0 0 0 0 0 ...
## $ AMOTSCO : num 0 0 0 0 0 0 0 0 0 0 ...
## $ AVRAAUT : num 0 0 0 0 0 0 0 0 0 0 ...
## $ AAANHANG: num 0 0 0 0 0 0 0 0 0 0 ...
## $ ATRACTOR: num 0 0 0 0 0 0 0 0 0 0 ...
## $ AWERKT : num 0 0 0 0 0 0 0 0 0 0 ...
## $ ABROM : num 0 0 0 0 0 0 0 1 0 0 ...
## $ ALEVEN : num 0 0 0 0 0 0 0 0 0 0 ...
## $ APERSONG: num 0 0 0 0 0 0 0 0 0 0 ...
## $ AGEZONG : num 0 0 0 0 0 0 0 0 0 0 ...
## $ AWAOREG : num 0 0 0 0 0 0 0 0 0 0 ...
## $ ABRAND : num 1 1 1 1 1 0 0 0 0 1 ...
## $ AZEILPL : num 0 0 0 0 0 0 0 0 0 0 ...
## $ APLEZIER: num 0 0 0 0 0 0 0 0 0 0 ...
## $ AFIETS : num 0 0 0 0 0 0 0 0 0 0 ...

```

```
## $ AINBOED : num 0 0 0 0 0 0 0 0 0 0 ...
## $ ABYSTAND: num 0 0 0 0 0 0 0 0 0 0 ...
## $ Purchase: Factor w/ 2 levels "No","Yes": 1 1 1 1 1 1 1 1 1 1 ...
```

```
summary(Purchase)
```

```
## No Yes
## 5474 348
```

```
348 / 5822 # only 6% purchased a caravan insurance!
```

```
## [1] 0.05977327
```

Because the KNN classifier predicts the class of a given test observation by identifying the observations that are nearest to it, the scale of the variables matters.

KNN works by measuring distances between points. If one variable ranges from 0-100 and another from 0-1, the first variable will dominate the distance calculation completely - even if it's not more important. `scale()` fixes this by giving every variable mean = 0 and sd = 1

```
standardized.X <- scale(Caravan[, -86]) #transforms every variable to have mean 0; sd 1.
# caravan data set, every row, every column BUT the 86th one (Purchase)
```

```
#by running the following two lines, we can see that the scales of two randomly
# chosen variables are very different!
```

```
var(Caravan[, 1])
```

```
## [1] 165.0378
```

```
var(Caravan[, 2])
```

```
## [1] 0.1647078
```

```
var(standardized.X[, 1]) #variance 1
```

```
## [1] 1
```

```
mean(standardized.X[, 1]) #mean approx. zero
```

```
## [1] 8.051119e-17
```

```
var(standardized.X[, 2])
```

```
## [1] 1
```

```
# Every column of standardized.X has a standard deviation of one and a mean of zero.
```

Now we build the KNN model.

```

# Split the observations into a test set and a training set
test <- 1:1000 # create a vector of row indices
train.X <- standardized.X[-test,] #takes all rows except test rows (from standardized data)
test.X <- standardized.X[test,] #takes all test rows (1-1000)
train.Y <- Purchase[-test] #same thing for Y (standardized.X doesnt contain Purchase)
test.Y <- Purchase[test]

set.seed(123)

knn.fit <- knn3(x = train.X, train.Y, k = 1)
# Fits the KNN model on the training data
# x = train.X : predictor matrix the model learns from
# train.Y : the true labels (Yes/No) for each training row
# k = 1 : use only the 1 nearest neighbor to make predictions
# -> for every new test point the model looks at the single closest
# training point and assigns its label
# eg new customer -> Find the 1 closest person in training data
# -> That person said "No" -> Predict "No"
# -> no real computation! It simply memorizes the entire training set

knn.pred <- predict(knn.fit, test.X, type = "class")
# predicts the class for each of the 1000 test observations
# knn.fit: trained model
# test.X: 1000 test observations to predict
# type = class: returns predicted class label, rather than probabilities

# Evaluate its performance on the test data
mean(test.Y != knn.pred) # computes the overall error rate of the KNN model on the test data

```

```
## [1] 0.117
```

```

# true value/predicted value same or different? -> compute error rate
# (number of wrong predictions/total predictions)
# Error rate on the test data is under 12 %

mean(test.Y != "No")

```

```
## [1] 0.059
```

```

# only 6 % of customers purchased insurance, we could get the error rate down to
# 6 % by always predicting No regardless of the values of the predictors!
# prediction error in naive model is lower than in KNN model

```

KNN tries to be smart and predict in both directions, but also makes errors in both directions. Error rate of 12%, worse than 6% (always saying no) Why is KNN doing worse?

Because the dataset is imbalanced: 94% No, 6% Yes. KNN gets confused by the rare “Yes” cases and also starts misclassifying some “No” cases, ending up with more total errors than the dumb baseline.

But does that mean the naive model is better? No! Because: The naive model never identifies a single buyer -> useless for business KNN at least tries to find buyers, even if imperfectly

```
table(knn.pred , test.Y) # confusion matrix: rows predicted, columns actual values
```

```
##           test.Y
## knn.pred  No Yes
##       No  873  49
##       Yes   68  10
```

```
10 / (68 + 10)
```

```
## [1] 0.1282051
```

```
# precision among predicted buyers: out of 78 predicted buyers, 10 will actually buy (12.8%)
```

```
# Using K =3
```

```
knn.fit_K3 <- knn3(train.X, train.Y, k = 3) # same procedure but with k=3!
#For every test point, look at the 3 closest neighbors and take a majority vote
knn.predict_K3 <- predict(knn.fit_K3, test.X, type = "class")
table(knn.predict_K3 , test.Y)
```

```
##           test.Y
## knn.predict_K3  No Yes
##              No  922  54
##              Yes   19   5
```

```
#KNN is now more conservative -> predicts Yes much less often
5 / (5 + 19)
```

```
## [1] 0.2083333
```

```
# but prediction quality improves: success rate increases to 21 %
```

```
# K = 5
```

```
knn.fit_K5 <- knn3(train.X, train.Y, k = 5) #k=5
knn.predict_K5 <- predict(knn.fit_K5, test.X, type = "class")
table(knn.predict_K5 , test.Y)
```

```
##           test.Y
## knn.predict_K5  No Yes
##              No  930  55
##              Yes   11   4
```

```
# as K increases, fewer people get flagged as buyers, higher precision among those flagged
4 / 15
```

```
## [1] 0.2666667
```

```
# rate is 26.7 %
# this strategy is cost-effective: cheaper to contact 15 high-quality leads than 78 random ones
# Context-specific! In health-related context
# -> recall would be more important (we don't want to miss any true positives)
```

KNN:

1. Standardize
2. Overall accuracy is misleading for imbalanced data
3. Confusion matrix & precision
4. K tradeoff: Higher K = fewer but better predictions

3)

Fit a Gaussian Process (GP) regression model to simulated data using the `gausspr()` function from the `kernlab` package and interpret the posterior mean and uncertainty.

What's a GP? -> A distribution over functions. Instead of predicting a single value like linear regression, it predicts 1) A mean: the best guess at each point 2) An uncertainty: how confident it is at each point -> plot: line with confidence band around it

Run the following code to generate the data.

```
library(kernlab)
```

```
##
```

```
## Attaching package: 'kernlab'
```

```
## The following object is masked from 'package:ggplot2':
```

```
##
```

```
##      alpha
```

```
library(ggplot2)
```

```
set.seed(42)
```

```
f_true <- function(x) sin(2 * pi * x) + 0.5 * sin(4 * pi * x) #true underlying function
```

```
n <- 40
```

```
x_train <- sort(runif(n, 0, 1)) # 40 random x values between 0 and 1
```

```
y_train <- f_true(x_train) + rnorm(n, sd = 0.3) #true function values plus random noise
```

```
x_grid <- seq(0, 1, length.out = 300)
```

```
#grid of 300 points between 0 and 1 on which GP will make predictions on
```

a) Fit and plot the GP posterior mean

Fit a GP regression model using `gausspr()` with a squared-exponential (RBF) kernel (`kernel = "rbfdot"`), `sigma = 5`, and `var = 0.09`. Predict on `x_grid` and plot the training data together with the posterior mean and the true function (dashed line).

```
gp_fit <- gausspr(x_train, y_train, kernel = "rbfdot",  
                  kpar = list(sigma = 5), var = 0.09)
```

```
# fits GP regression model. training data
```

```
# (input(what we know)/output(what we want to predict))
```

```
# specify kernel function (gaussian; default choice)
```

```
# we look at kpar and var in c and d
```

```
# gp_fit then stores the fitted GP model
```

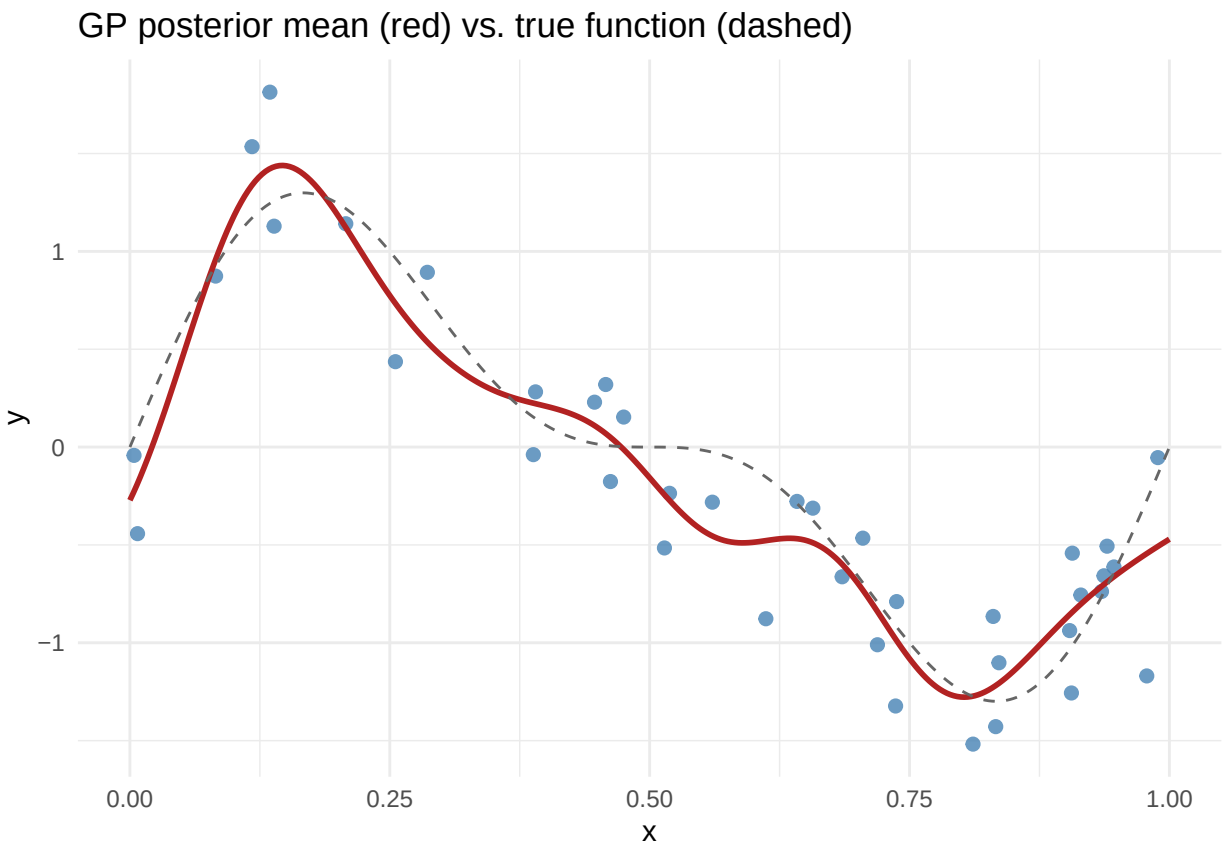
```

y_pred <- predict(gp_fit, x_grid)
# uses the trained GP model to predict the posterior mean at every point in x_grid
# for every point in x_grid, the GP gives its best guess for y
# all values are stored in y

df_train <- data.frame(x = x_train, y = y_train)
# combines x and y training data into data frame
df_grid <- data.frame(x = x_grid, y_pred = as.vector(y_pred))
# data frame with x grid points and predicted values of y

ggplot() +
  geom_point(data = df_train, aes(x, y),
            colour = "steelblue", size = 2, alpha = 0.8) +
  geom_line(data = df_grid, aes(x, y_pred),
            colour = "firebrick", linewidth = 1) +
  geom_line(aes(x = x_grid, y = f_true(x_grid)),
            linetype = "dashed", colour = "grey40") +
  labs(title = "GP posterior mean (red) vs. true function (dashed)",
       x = "x", y = "y") +
  theme_minimal()

```



dots: 40 training points (noisy observations) red line: GP's best guess / posterior mean dashed line: true underlying function (noiseless) which the GP is trying to recover

b) Obtain and plot posterior uncertainty

Refit the GP with `variance.model = TRUE`. Use `predict(..., type = "sdeviation")` to extract posterior standard deviations and add ± 1.96 credible bands to your plot. Where is uncertainty largest, and why?

```
gp_var <- gausspr(x_train, y_train, kernel = "rbfdot",
                  kpar = list(sigma = 5), var = 0.09,
                  variance.model = TRUE)
#same code as in a) but it tells the GP to also track and return uncertainty
# at each point, not just the mean

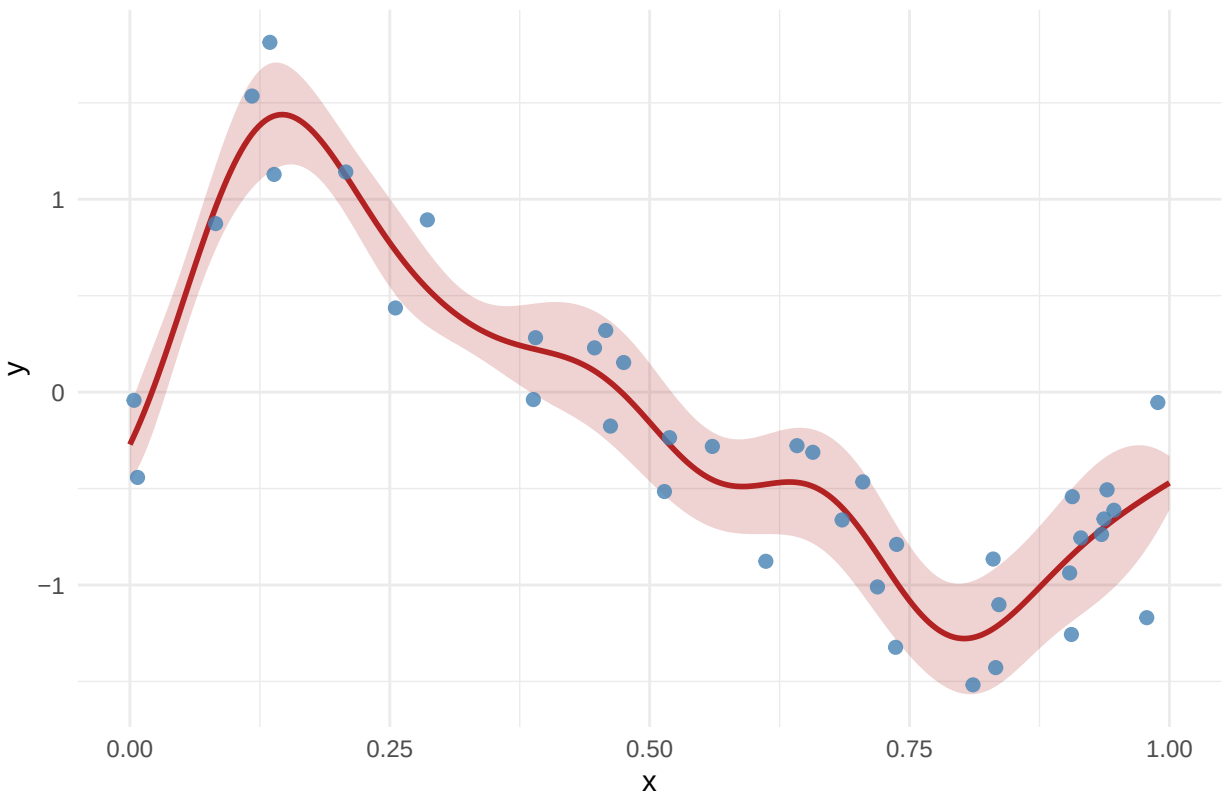
y_mean <- as.vector(predict(gp_var, x_grid))
#extracts posterior mean

y_sd <- as.vector(predict(gp_var, x_grid, type = "sdeviation"))
# extracts posterior standard deviation at each grid point - how uncertain GP is

df_band <- data.frame( # computes data frame with:
  x = x_grid, # x grid points
  mu = y_mean, # their respective predicted mean
  lo = y_mean - 1.96 * y_sd, # lower bound of band = mean - 1.96 * sd
  hi = y_mean + 1.96 * y_sd # upper bound of band = mean + 1.96 * sd
)

ggplot() +
  geom_ribbon(data = df_band, aes(x = x, ymin = lo, ymax = hi),
            fill = "firebrick", alpha = 0.2) +
  geom_line(data = df_band, aes(x = x, y = mu),
            colour = "firebrick", linewidth = 1) +
  geom_point(data = df_train, aes(x, y),
             colour = "steelblue", size = 2, alpha = 0.8) +
  labs(title = "GP posterior mean with 95% credible band",
       x = "x", y = "y") +
  theme_minimal()
```

GP posterior mean with 95% credible band



The shaded band shows $\hat{\mu}(x) \pm 1.96 \hat{\sigma}(x)$.

Uncertainty is smallest near training points (GP has seen data there and is confident)

The band widens both where data is sparse and where observations scatter far from the posterior mean (GP has less confidence about where the true function lies)

This calibrated uncertainty is a key advantage of GPs over point-prediction methods such as simple regression.

c) Effect of the length-scale hyperparameter

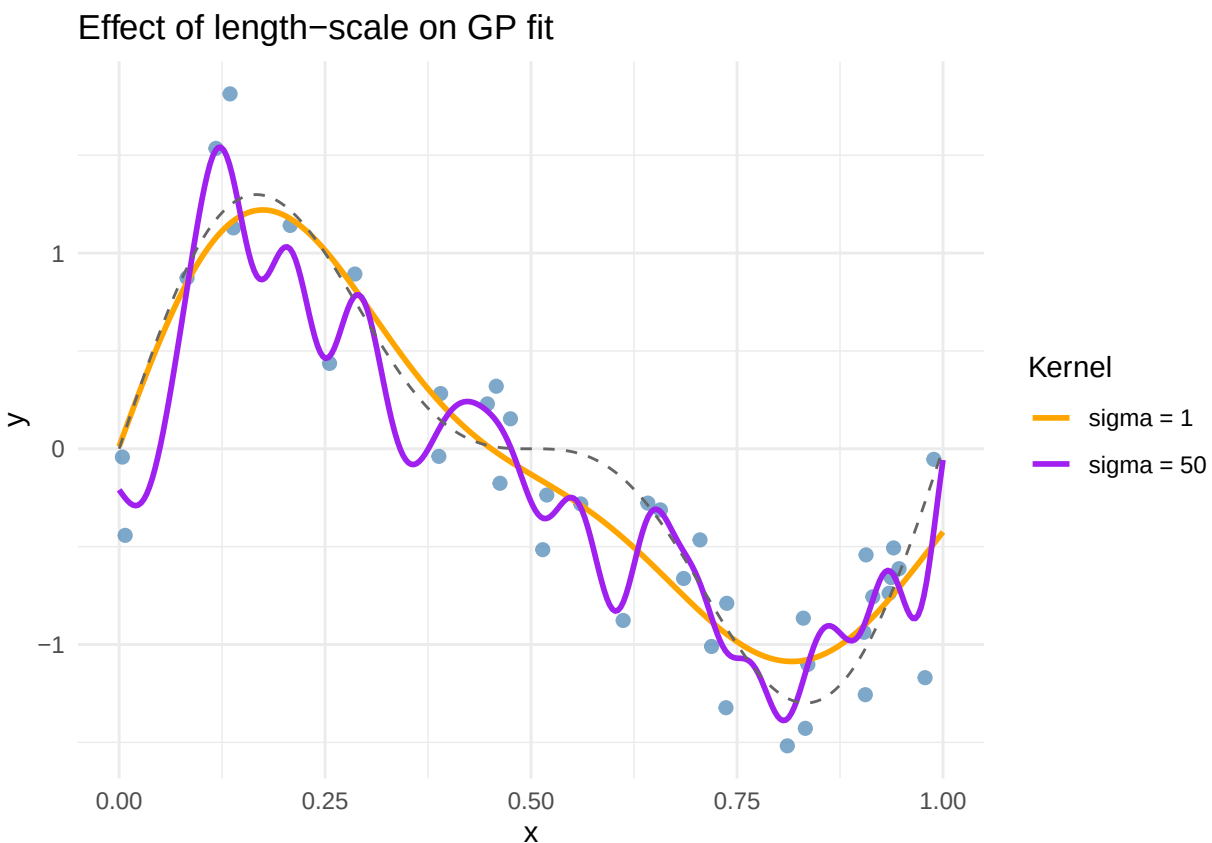
Refit the GP with `sigma = 1` and `sigma = 50` (keeping `var = 0.09`). Plot both posterior means together with the data and the true function. Describe the effect of `sigma` on the fit.

```
gp_s1 <- gausspr(x_train, y_train, kernel = "rbfdot",
                 kpar = list(sigma = 1), var = 0.09)
gp_s50 <- gausspr(x_train, y_train, kernel = "rbfdot",
                  kpar = list(sigma = 50), var = 0.09)

df_comp <- data.frame(
  x      = x_grid,
  sigma1 = as.vector(predict(gp_s1, x_grid)),
  sigma50 = as.vector(predict(gp_s50, x_grid))
)
```



```
ggplot() +
  geom_point(data = df_train, aes(x, y),
            colour = "steelblue", size = 2, alpha = 0.7) +
  geom_line(data = df_comp,
            aes(x, sigma1, colour = "sigma = 1"), linewidth = 1) +
  geom_line(data = df_comp,
            aes(x, sigma50, colour = "sigma = 50"), linewidth = 1) +
  geom_line(aes(x = x_grid, y = f_true(x_grid)),
            linetype = "dashed", colour = "grey40") +
  scale_colour_manual(values = c("sigma = 1" = "orange",
                                "sigma = 50" = "purple")) +
  labs(title = "Effect of length-scale on GP fit",
       colour = "Kernel", x = "x", y = "y") +
  theme_minimal()
```



In `kernlab`'s `rbfdot`, the kernel is $k(x, x') = \exp(-\sigma \|x - x'\|^2)$. The parameter `sigma` controls the length-scale: how far apart two points can be before they stop being correlated with each other

- **Small sigma** (e.g. 1): long length-scale: the influence of each training point reaches far. Even distant points are still considered correlated, so the GP must change slowly. Produces a very smooth, almost flat posterior mean that under-fits the data.
- **Large sigma** (e.g. 50): short length-scale: the influence of each training point reaches only its immediate neighbors. Distant points are uncorrelated, so the GP can change rapidly. Wiggly fit of the curve, over-fits the noise
- `sigma = 5` strikes a reasonable balance for this dataset.

This is analogous to bandwidth in KDE: - small bandwidth $\rightarrow$ large sigma $\rightarrow$ wiggly - large bandwidth $\rightarrow$ small sigma $\rightarrow$ over-smoothed

In practice, hyperparameters are selected by maximizing the marginal likelihood (type-II maximum likelihood).

d) Effect of the noise variance

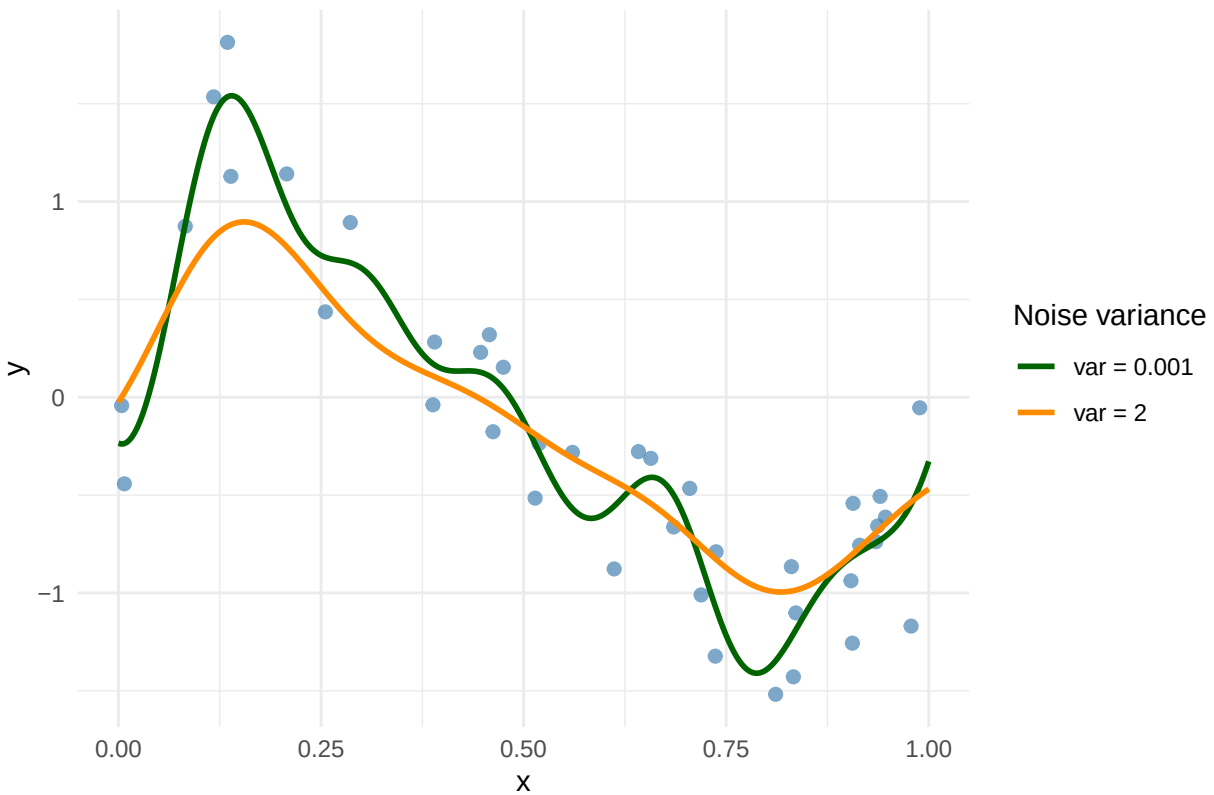
Using `sigma = 5`, refit the GP with `var = 0.001` and `var = 2`. Plot the two posterior means. What does the noise variance parameter control, and how does changing it affect the fit?

```
gp_v001 <- gausspr(x_train, y_train, kernel = "rbfdot",
                  kpar = list(sigma = 5), var = 0.001)
gp_v2    <- gausspr(x_train, y_train, kernel = "rbfdot",
                  kpar = list(sigma = 5), var = 2)

df_noise <- data.frame(
  x      = x_grid,
  v001   = as.vector(predict(gp_v001, x_grid)),
  v2     = as.vector(predict(gp_v2,   x_grid))
)

ggplot() +
  geom_point(data = df_train, aes(x, y),
            colour = "steelblue", size = 2, alpha = 0.7) +
  geom_line(data = df_noise,
            aes(x, v001, colour = "var = 0.001"), linewidth = 1) +
  geom_line(data = df_noise,
            aes(x, v2,   colour = "var = 2"),      linewidth = 1) +
  scale_colour_manual(values = c("var = 0.001" = "darkgreen",
                                "var = 2"     = "darkorange")) +
  labs(title = "Effect of noise variance on GP fit",
       colour = "Noise variance", x = "x", y = "y") +
  theme_minimal()
```

Effect of noise variance on GP fit



The `var` parameter is the assumed observation noise variance σ_n^2 added to the diagonal of the kernel matrix.

- **Small var** (0.001): the GP assumes the data are nearly noiseless, so the posterior mean interpolates through every training point, producing a wiggly fit that over-fits the noise.
- **Large var** (2): the GP assumes very noisy data, so the posterior mean is heavily smoothed and ignores individual observations, it rather follows a general trend.

The true noise variance here is $0.3^2 = 0.09$, which is why `var` = 0.09 gives the best-calibrated fit.